

VITyarthi BYOP: GitHub Repository README

Course: Programming in Java (CSE2006) | Student: Jainam Jain (Reg: 25BA111474)

README.MD

Project Document: README.md

1. Project Title & Overview

Hangman Game in Java is an Object-Oriented console puzzle game built to demonstrate core Java principles and clean modular architecture. Instead of bundling execution logic into a single file, the system is separated into discrete classes for user interaction, state management, dictionary storage, defensive input validation, and local file I/O. The game provides difficulty-based word guessing, turn tracking, and persistent session scoring.

2. Key Features

Three Difficulty Levels

Custom word lists across Easy (7 turns), Medium (5 turns), and Hard (4 turns) with random word selection via `java.util.Random`.

Robust Input Filtering

Blocks digits, whitespace, and symbols. Alerts the player upon duplicate entries without deducting remaining attempts.

Score Persistence

Calculates outcome points (`Attempts Remaining × 10`) upon victory and appends player records to `scores.txt`.

Automated Verification

Includes a standalone test suite (`TestGame.java`) verifying masking, state changes, and input validation without interactive gameplay.

3. Technologies & Tools Used

Java Standard Edition (JDK 8+)

OOP Design (Classes & Objects)

Encapsulation & Accessors

File I/O (FileWriter / Scanner)

Defensive Validation

Git Version Control

VS Code / Terminal CLI

4. Project Structure

The repository is organized into distinct, modular Java source files fulfilling the academic project requirement:

Hangman/ |—— Main.java # Entry point, interactive console menu, and game loop |—— Game.java # Game e

5. Steps to Install & Run

Follow these commands to clone, compile, and execute the application using the command line:

```
# 1. Clone repository from GitHub git clone <your-github-repository-url> cd hangman-java # 2. Compile
```

6. Instructions for Testing (TestGame.java)

An automated test driver is included to verify core mechanics and validation algorithms independently:

```
# Compile and execute the test harness javac TestGame.java java TestGame
```

Test Case	Target Functionality	Expected Result	Status
Test 1	Word Masking Verification	Unrevealed word displays strictly as underscores	PASS
Test 2	Valid Character Guess	Correct letter replaces underscore; attempts unchanged	PASS
Test 3	Incorrect Character Guess	Remaining attempts decrease by exactly 1	PASS
Test 4	Input Sanitization Check	Numbers, symbols, and blanks are blocked with zero penalty	PASS

7. Sample Console Output

```
What's your name? Alex Hi, Alex Time to play Hangman! 1. Game Time | 2. See Scores | 3. Exit Game Mak
```